Chapter 11. Building with LLMs

One of the biggest open questions in the world of LLMs today is how to best put them in the hands of end users. In some ways, LLMs are actually a more intuitive interface for computing than what came before them. They are much more forgiving of typos, slips of the tongue, and the general imprecision of humans, when compared to traditional computer applications. On the other hand, the very ability to handle inputs that are "slightly off" comes with a tendency to sometimes produce results that are also "slightly off"—which is also very much unlike any previous computing tendencies.

In fact, computers were designed to reliably repeat the same set of instructions with the same results every time. Over the past few decades, that principle of reliability has permeated the design of human-computer interfaces (variously called HCI, UX, and UI) to the extent that a lot of the usual constructs end up being subpar for use in applications that rely heavily on LLMs.

Let's take an example: Figma is a software application used by designers to create faithful renderings of designs for websites, mobile applications, book or magazine covers—the list goes on. As is the case with pretty much all productivity software (software for the creation of some kind of long-form content), its interface is a combination of the following:

- A palette of tools and prebuilt *primitives* (fundamental building blocks), in this case lines, shapes, selection and paint tools, and many more
- A canvas, where the user inserts these building blocks and organizes them into their creation: a website page, a mobile app screen, and so on

This interface is built upon the premise that the capabilities of the software are known ahead of time, which is in fact true in the case of Figma. All building blocks and tools were coded by a software engineer ahead of

time. Therefore, they were known to exist at the time the interface was designed. It sounds almost silly to point that out, but the same is not strictly true of software that makes heavy use of LLMs.

Look at a word processor (e.g., Microsoft Word or Google Docs). This is a software application for the creation of long-form text content of some kind, such as a blog post, article, book chapter, and the like. The interface at our disposal here is also made up of a familiar combination:

- A palette of tools and prebuilt *primitives*: in the case of a word processor, the primitives available are tables, lists, headings, image placeholders, and so forth, and the tools are spellcheck, commenting, and so on.
- *A canvas*: in this case, it's literally a blank page, where the user types words and may include some of the elements just mentioned.

How would this situation change if we were to build an LLM-native word processor? This chapter explores three possible answers to this question, which are broadly applicable to any LLM application. For each of the patterns we explore, we'll go over what key concepts you'd need to implement it successfully. We don't mean to imply that these are the only ones, it will be a while until the dust settles on this particular question.

Let's look at each of these patterns, starting with the easiest to add to an existing app.

Interactive Chatbots

This is arguably the easiest lift to add to an existing software application. At its most basic conception, this idea just bolts on an AI sidekick—to bounce ideas off of—while all work still happens in the existing user interface of the application. An example here is GitHub Copilot Chat, which can be used in a sidebar inside the VSCode code editor.

An upgrade to this pattern is to add some communication points between the AI sidekick extension and the main application. For example, in VSCode, the assistant can "see" the content of the file currently being edited or whatever portion of that code the user has selected. And in the oth-

er direction, the assistant can insert or edit text in that open editor, arriving at some basic form of collaboration between the user and the LLM.

NOTE

Streaming chat as we're describing here is currently the prototypical application of LLMs. It's almost always the first thing app developers learn to build on their LLM journey, and it's almost always the first thing companies reach for when adding LLMs to their existing applications. Maybe this will remain the case for years to come, but another possible outcome could be for streaming chat to become the command line of the LLM era—that is, the closest to direct programming access, becoming a niche interface, just as it did for computers.

To build the most basic chatbot you should use these components:

A chat model

Their dialogue tuning lends itself well to multiturn interactions with a user. Refer to the Preface for more on dialogue tuning.

Conversation history

A useful chatbot needs to be able to "get past hello." That is, if the chatbot can't remember the previous user inputs, it will be much harder to have meaningful conversations with it, which implicitly refer to previous messages.

To go beyond the basics, you'd probably add the following:

Streaming output

The best chatbot experiences currently stream LLM output token by token (or in larger chunks, like sentences or paragraphs) directly to the user, which alleviates the latency inherent to LLMs today.

Tool calling

To give the chatbot the ability to interact with the main canvas and tools of the application, you can expose them as tools the model can decide to call on—for instance, a "get selected text" tool and an "insert text at end of doc" tool.

Human-in-the-loop

As soon as you give the chatbot tools that can change what's in the application canvas, you create the need to give back some control to the user—for example, letting the user confirm, or even edit, before new text is inserted.

Collaborative Editing with LLMs

Most productivity software has some form of collaborative editing built in, which we can classify into one of these buckets (or somewhere in between):

Save and send

This is the most basic version, which only supports one user editing the document at a time, before "passing the buck" to another user (for example, sending the file over email) and repeating the process until done. The most obvious example is the Microsoft Office suite of apps: Excel, Word, PowerPoint.

Version control

This is an evolution of save and send that supports multiple editors working simultaneously on their own (and unaware of each other's changes) by providing tools to combine their work afterward: merge strategies (how to combine unrelated changes) and conflict resolution (how to combine incompatible changes). The most popular example today is Git/GitHub, used by software engineers to collaborate on software projects.

Real-time collaboration

This enables multiple editors to work on the same document at the same time, while seeing each other's changes. This is arguably the most natural form of software-enabled collaboration, evidenced by the popularity of Google Docs and Google Sheets among technical and nontechnical computer users.

This pattern of LLM user experience consists of employing an LLM agent as one of those "users" contributing to this shared document. This can take many forms, including the following:

- An always-on “copilot” giving you suggestions on how to complete the next sentence
- An asynchronous “drafter,” which you task with, for example, going off and researching the topic in question and returning later with a section you can incorporate in your final document

To build this, you’d likely need the following:

Shared state

The LLM agent and the human users should be on the same footing in terms of access and understanding of the state of the document—that is, they would be able to parse the state of the document and produce edits to that state in a compatible format.

Task manager

Producing a useful edit to the document will invariably be a multi-step process, which can take time and fail halfway. This creates the need for reliable scheduling and orchestration of long-running jobs, with queueing, error recovery, and control over running tasks.

Merging forks

Users will continue to edit the document after tasking the LLM agent, so LLM outputs will need to be merged with the users’ work, either manually by the user (an experience like Git) or automatically (through conflict resolution algorithms such as CRDT and operational transformation (OT), employed by applications such as Google Docs).

Concurrency

The fact that the human user and the LLM agent are working on the same thing at the same time requires the ability to handle interruptions, cancellations, reroutings (do this instead), and queueing (do this as well).

Undo/redo stack

This is a ubiquitous pattern in productivity software, which inevitably is needed here too. Users change their minds and want to

go back to an earlier state of the document, and the LLM application needs to be capable of following them there.

Intermediate output

Merging user and LLM outputs is made a lot easier when those outputs are gradual and arrive piecemeal as soon as they're produced, in much the same way that a person writes a 10-paragraph page one sentence at a time.

Ambient Computing

A very useful UX pattern has been the always-on background software that pipes up when something "interesting" has happened that deserves your attention. You can find this in many places today. A few examples are:

- You can set an alert in your brokerage app to notify you when some stock goes below a certain price.
- You can ask Google to notify you when new search results are found matching some search query.
- You can define alerts for your computer infrastructure to notify you when something is outside the regular pattern of behavior.

The main obstacle to deploying this pattern more widely may be coming up with a reliable definition of *interesting* ahead of time that is both of the following:

Useful

It will notify you when you think it should.

Practical

Most users won't want to spend massive amounts of time ahead precreating endless rules for alerts.

The reasoning capabilities of LLMs can unlock new applications of this pattern of *ambient computing* that are simultaneously more useful (they identify more of what you'd find interesting) and less work to set up (their reasoning can replace a lot or all of the manual setup of rules).

The big difference between *collaborative* and *ambient* is concurrency:

Collaborative

: You and the LLM are usually (or sometimes) doing work at the same time and feeding off each other's work.

Ambient

: The LLM is continuously doing some kind of work in the background while you, the user, are presumably doing something else entirely.

To build this, you need:

Triggers

: The LLM agent needs to receive (or poll periodically for) new information from the environment. This is in fact what motivates ambient computing: a preexisting source of periodic or continuous new information that needs to be processed.

Long-term memory

: It would not be possible to detect new interesting events without consulting a database of previously received information.

Reflection (or learning)

: Understanding what is *interesting* (what deserves human input) likely requires learning from each previous interesting event after it happens. This is usually called a *reflection step*, in which the LLM produces an update to its long-term memory, possibly modifying its internal "rules" for detecting future interesting events.

Summarize output

: An agent working in the background is likely to produce much more output than the human user would like to see. This requires that the agent architecture be modified to produce summaries of the work done and surface to the user only what is new or noteworthy.

Task manager

> Having an LLM agent working continuously in the background requires employing some system for managing the work, queuing new runs, and handling and recovering from error.

Summary

LLMs have the potential to change not only how we build software, but also the very software we build. This new capability that we developers have at our disposal to generate new content will not only enhance many existing apps, but it can make new things possible that we haven't dreamed of yet.

There's no shortcut here. You really do need to build something (s)crappy, speak to users, and rinse and repeat until something new and unexpected comes out the other side.

With this last chapter, and the book as a whole, we have tried to give you the knowledge we think can help you build something uniquely good with LLMs. We want to thank you for coming on this journey with us and wish you the best of luck in your career and future.